

Rust Ownership and Borrowing

Based on ECM2433 Rust ownership, slices/strings, and lifetime slides.

Description: What You Need To Know

1. The ownership rules

- Each value in Rust has an owner.
- There can only be one owner at a time.
- When the owner goes out of scope, the value is dropped.

2. Stack, heap, and String

Fixed-size scalar values often live on the stack. A String owns heap text; its stack part stores a pointer, length, and capacity.

3. Move, Copy, and clone

```
let s1 = String::from("hello");
let s2 = s1;           // move
// println!("{s1}"); // error: s1 was moved

let x = 5;
let y = x;             // Copy
println!("{x} {y}");
```

4. Borrowing with references

A reference borrows a value without taking ownership. For read-only text parameters, prefer `&str` because it accepts string literals and borrowed String values.

```
fn announce(message: &str) {
    println!("{message}");
}

let msg = String::from("hello");
announce(&msg);
println!("{msg}");
```

5. Mutable references and borrowing rules

A mutable reference allows modification through a borrow. Rust allows either one mutable reference or any number of immutable references, but not both at the same time.

```
fn shout(text: &mut String) {
    text.push_str("!");
}

let mut msg = String::from("hello");
shout(&mut msg);
```

6. Slices and lifetimes

`&[T]` is a borrowed slice of elements. `&str` is a borrowed string slice. Rust rejects references that could outlive the data they point to.

```
fn bad() -> &str {
    let s = String::from("hello");
    &s
}

fn good() -> String {
    String::from("hello")
}
```

Practice Questions

- 1 Write the three ownership rules in your own words.
- 2 Explain why `let s2 = s1; println!("{s1}");` fails when `s1` is a String.
- 3 Explain why copying an `i32` leaves both variables usable.
- 4 Fix a function parameter so it can print read-only text without taking ownership from the caller.
- 5 Write a `count_vowels` signature using `&str` and explain why that parameter type is useful.

- 6 Explain why two active mutable references to the same `String` are not allowed.
- 7 Write a function that appends `!` to a `String` through a mutable reference.
- 8 Explain the difference between `String` and `&str`.
- 9 Explain why a function cannot return a reference to a `String` created inside that function.
- 10 For `let nums = [10, 20, 30, 40];`, show how to pass the whole array to a function taking `&[i32]`.

Mark Scheme

Use this after attempting the questions.

- 1 Each value has one owner; only one owner exists at a time; when the owner goes out of scope, the value is dropped.
- 2 String owns heap data. Assignment moves ownership to s2, so s1 is no longer valid.
- 3 i32 implements Copy, so assignment duplicates the value rather than moving ownership.
- 4 Prefer `fn announce(message: &str)`, then call it with a literal or a borrowed String.
- 5 Expected shape: `fn count_vowels(text: &str) -> usize`. It only reads text and accepts literals and borrowed String values.
- 6 Two active mutable references could both write to the same value. Rust enforces one writer or many readers.
- 7 Expected shape: `fn shout(text: &mut String) { text.push_str("!"); }`, called with a mutable borrow.
- 8 String owns growable heap text. `&str` is a borrowed string slice: pointer plus length, no ownership.
- 9 The local String is dropped when the function returns, so the reference would point to invalid data. Return an owned String instead.
- 10 Expected: call a function like `use_all(&nums)` when its parameter is `&i32`.